M92U0人形20241202版本URDF检查
1. M92U0模型版本
2. 惯性参数检查
2.1 惯性算法检查
2.2 可视化检查
2.2.1 惯性矩阵可视化检查
2.2.2 质量的可视化检查
2.2.3 灵巧手常用惯性参数估算方法
3. 建议优化的内容
3.1 灵巧手相关坐标系
3.2 灵巧手控制方向的问题
附录（其它人形机器人可视化）：

M92U0人形20241202版本URDF检查

李向远 | 2024年12月14日创建

1. M92U0模型版本

> 硬件组提供的模型信息：
>
> 根据结构组20241202输出的M92U0人形的creo模型，更新了M92U0人形机器人的初版xacro文件，包括整机的运动学参数、电机参数、质量惯量参数以及stl文件，其中质量参数为creo模型的预估质量，模型总质量：71.9kg。
> 已更新到https://git.n.xiaomi.com/mirobotics_sim/m92u/m92_u_description/-/tree/master的master分支。

问题概述：

- 关节、连杆还未同步到最新的命名规范。
- 有8个 `Link` 的惯性参数不合规。
- 灵巧手的惯性参数明显与实际不符，手指质量分布过小。
- 头和手腕的惯性参数可能与实际不符。
- 腰部（实际上应该是胸腔）的惯性矩阵有倾斜，需核实密度是否均匀。

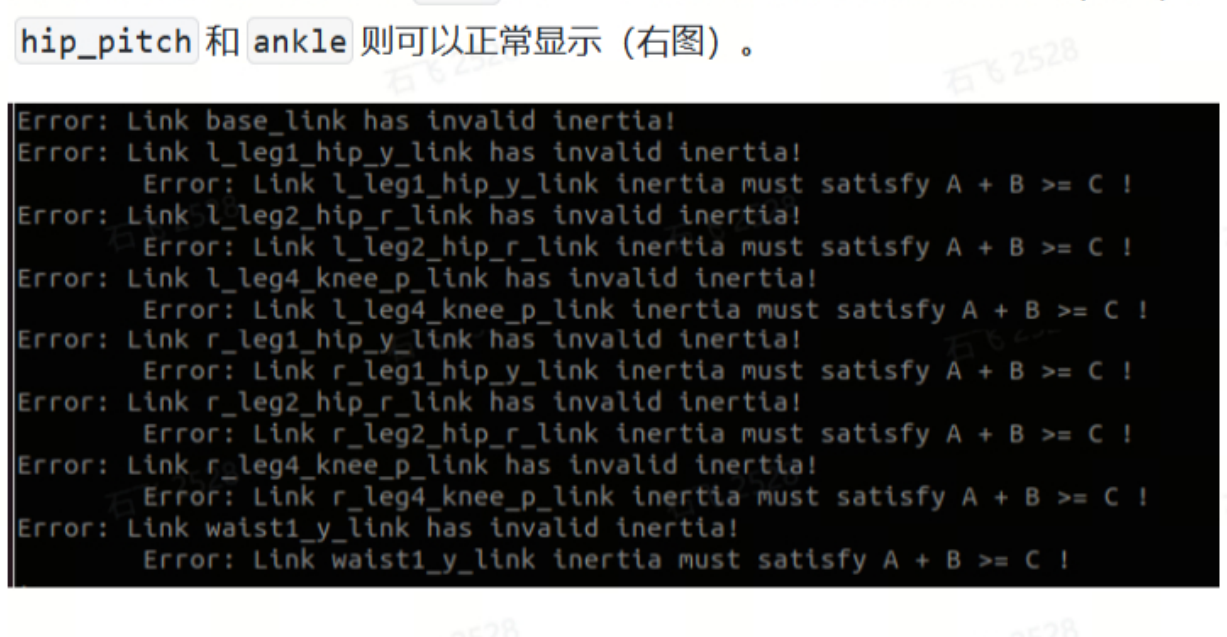
建议优化的地方：

- 将腰部 `pitch` 的 `Link` 拆解成 `waist2_p_link` 和 `torso_link` 。
- 不要将灵巧手的手掌作为 `l_arm7_wrist_r_link` 的一部分。
- 增加手腕TCP坐标系和手的末端执行器坐标系。
- 增加足底前、后端辅助坐标系： `foot_forefoot` 和 `foot_heel` 。
- 灵巧手控制方向的优化。
- Mesh文件缩放的问题，当前Mesh文件使用时需缩小1000倍，当与SDF搭配使用时有可能穿模，建议不要缩放。
- 除灵巧手关节外，其它关节的动力学参数全部为0，建议可参考灵巧手，赋一个预估值。

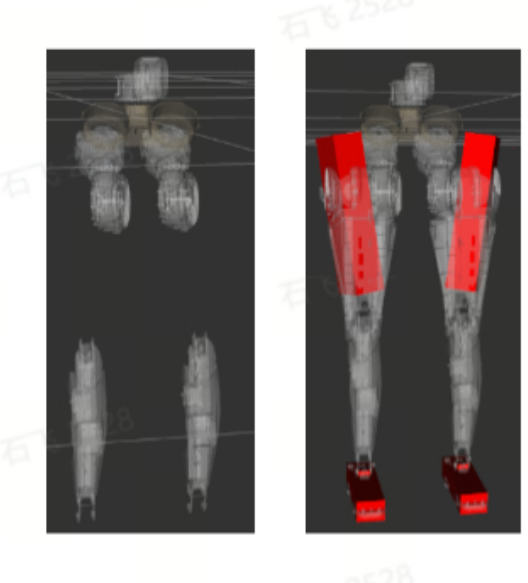
2. 惯性参数检查

2.1 惯性算法检查

根据URDF惯性参数检查方法中提到的检查方法，发现共有8个 `Link` 的惯性参数不合规（左图）。主要分布在基座、腿和腰上，这8个 `Link` 在Rviz中匀无法正常显示惯性矩阵框（中图），同时，惯性参数正常的 `hip_pitch` 和 `ankle` 则可以正常显示（右图）。



```
Error: Link base_link has invalid inertia!
Error: Link l_leg1_hip_y_link has invalid inertia!
        Error: Link l_leg1_hip_y_link inertia must satisfy A + B >= C !
Error: Link l_leg2_hip_r_link has invalid inertia!
        Error: Link l_leg2_hip_r_link inertia must satisfy A + B >= C !
Error: Link l_leg4_knee_p_link has invalid inertia!
        Error: Link l_leg4_knee_p_link inertia must satisfy A + B >= C !
Error: Link r_leg1_hip_y_link has invalid inertia!
        Error: Link r_leg1_hip_y_link inertia must satisfy A + B >= C !
Error: Link r_leg2_hip_r_link has invalid inertia!
        Error: Link r_leg2_hip_r_link inertia must satisfy A + B >= C !
Error: Link r_leg4_knee_p_link has invalid inertia!
        Error: Link r_leg4_knee_p_link inertia must satisfy A + B >= C !
Error: Link waist1_y_link has invalid inertia!
        Error: Link waist1_y_link inertia must satisfy A + B >= C !
```

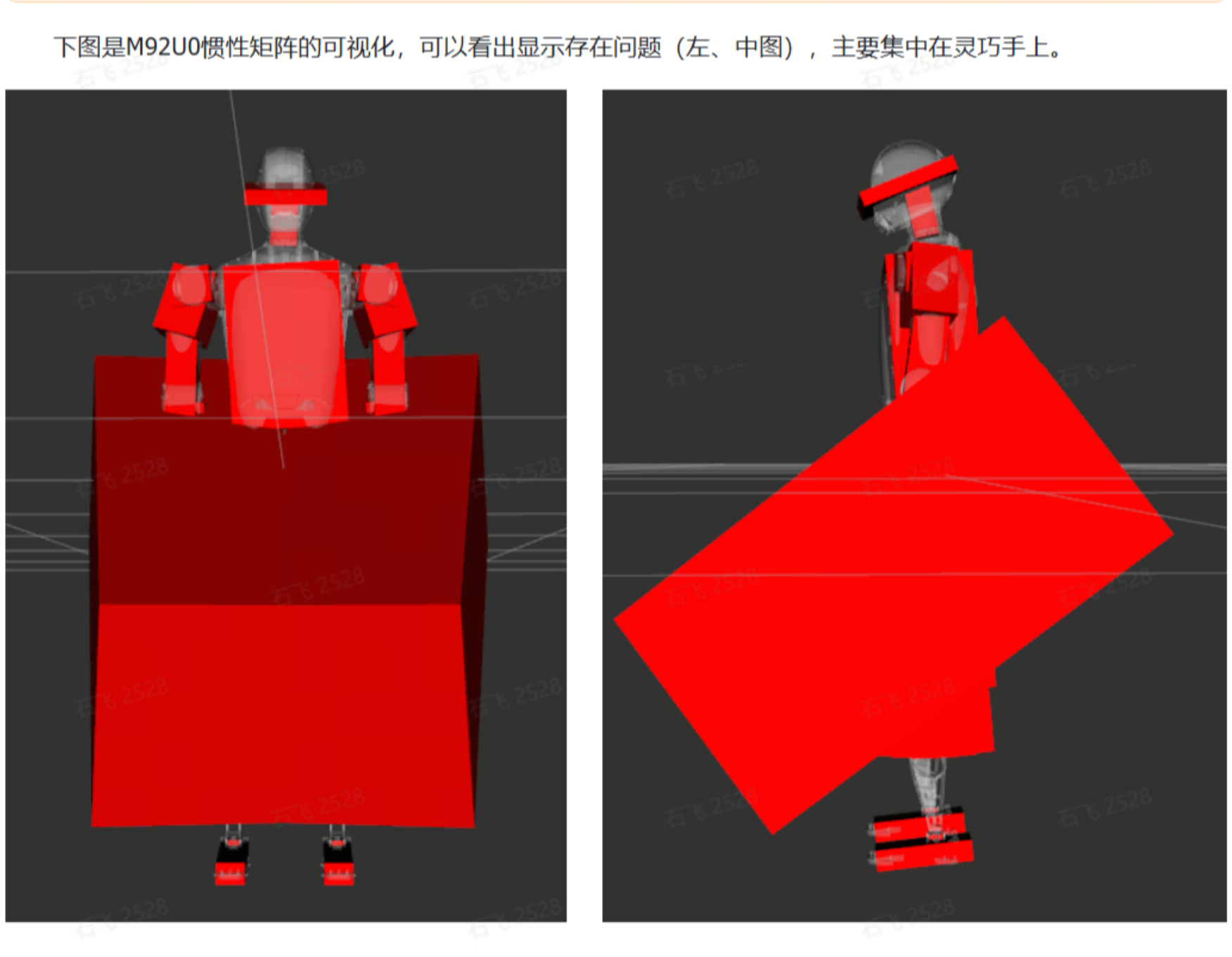


2.2 可视化检查

2.2.1 惯性矩阵可视化检查

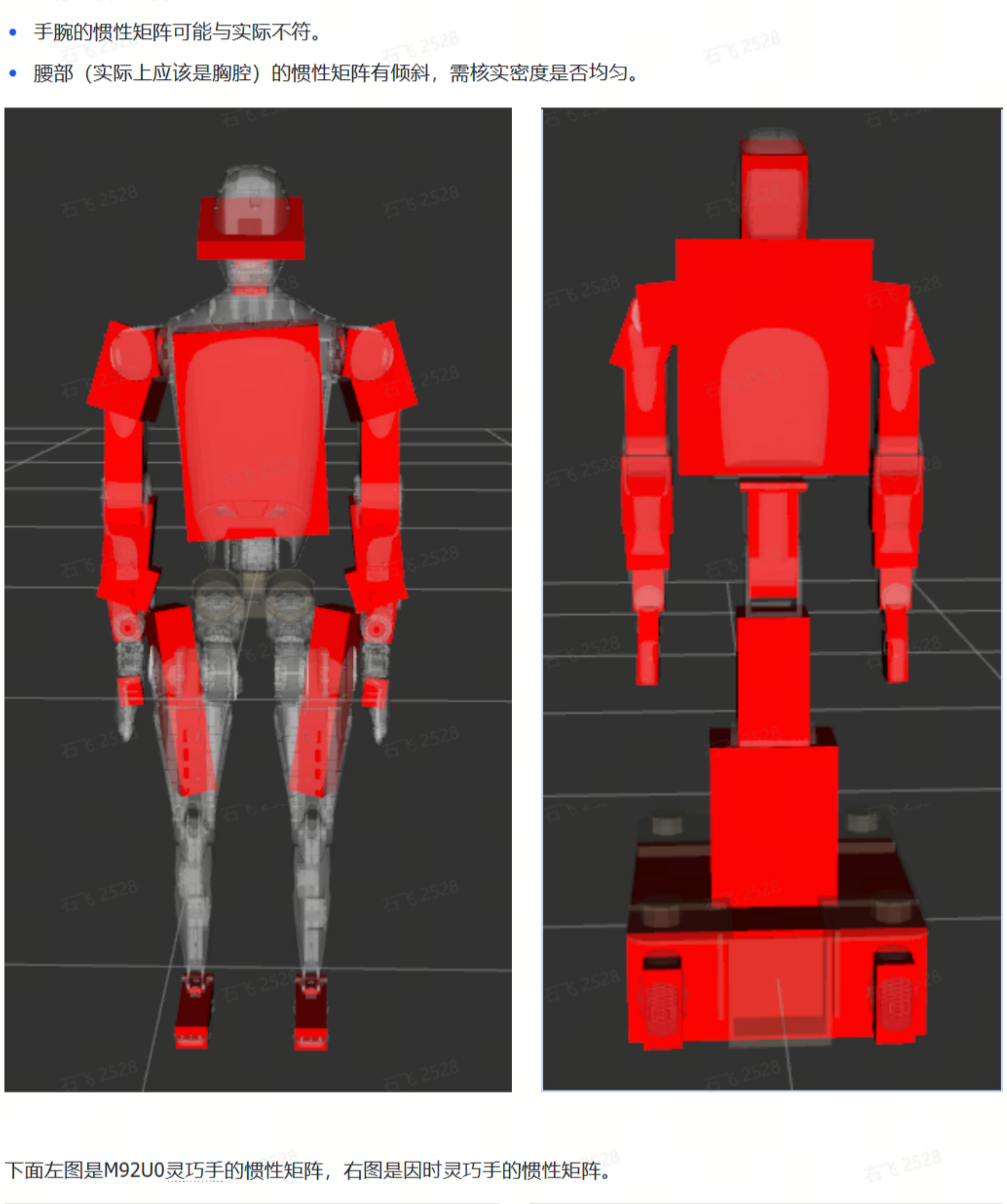
> 第一步检查出的8个惯性参数异常的 `Link` ，无法显示惯性矩阵框，所以这里检查的主要是能通过算法检查的其它 `Link` 。

下图是M92U0惯性矩阵的可视化，可以看出显示存在问题（左、中图），主要集中在灵巧手上。

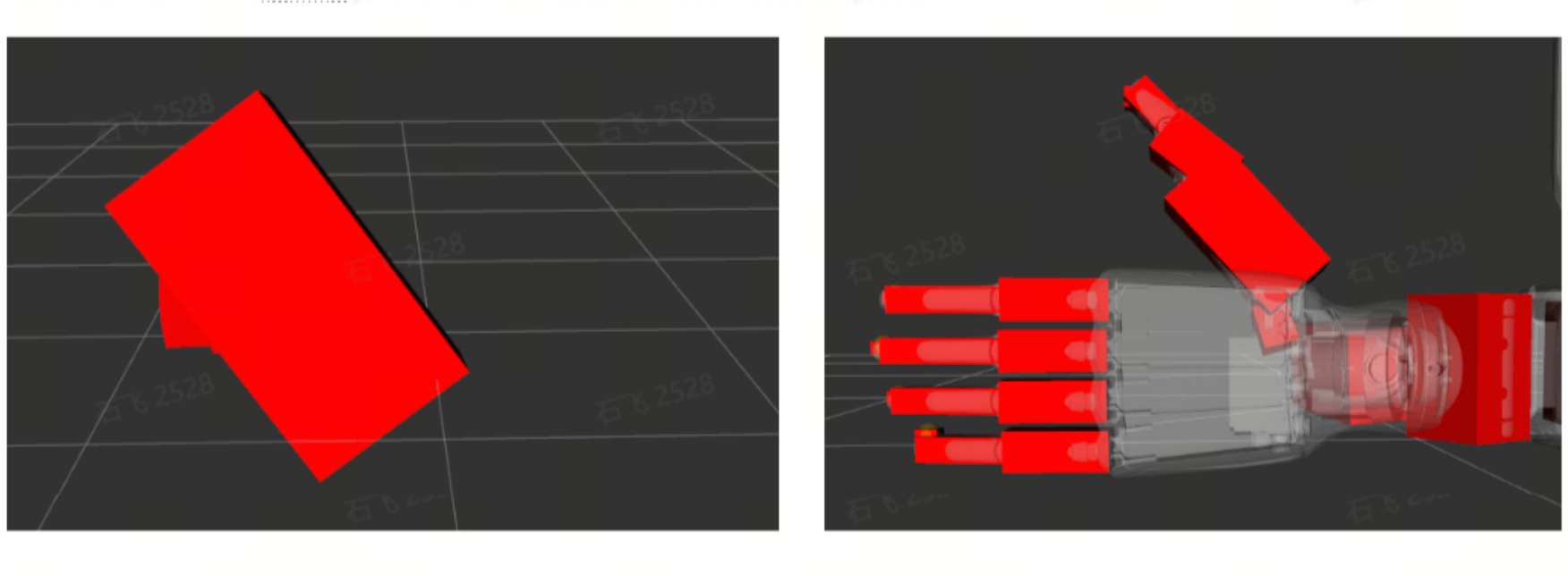


下面左图是去掉灵巧手后的惯性矩阵，右图是M92U轮式的惯性矩阵，对比可以发现：

- 头部的惯性矩阵可能与实际不符。
- 手腕的惯性矩阵可能与实际不符。
- 腰部（实际上应该是胸腔）的惯性矩阵有倾斜，需核实密度是否均匀。

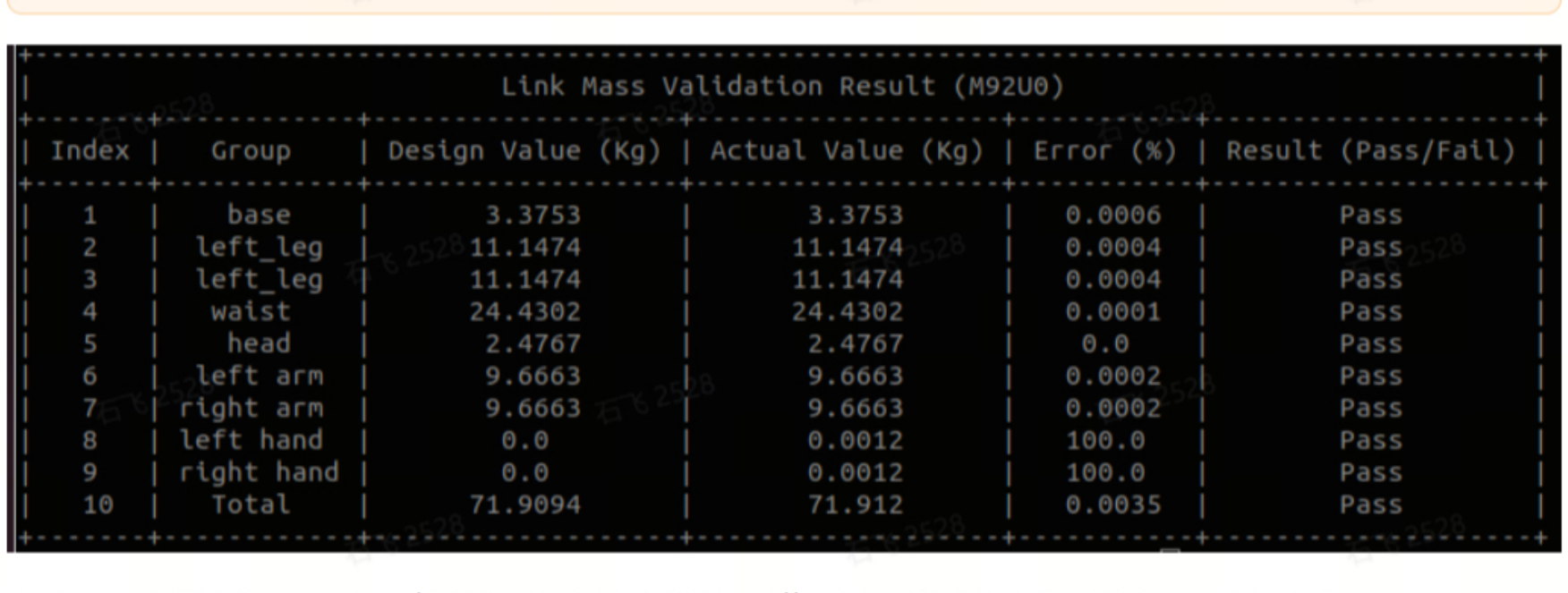


下面左图是M92U0灵巧手的惯性矩阵，右图是因时灵巧手的惯性矩阵。



2.2.2 质量的可视化检查

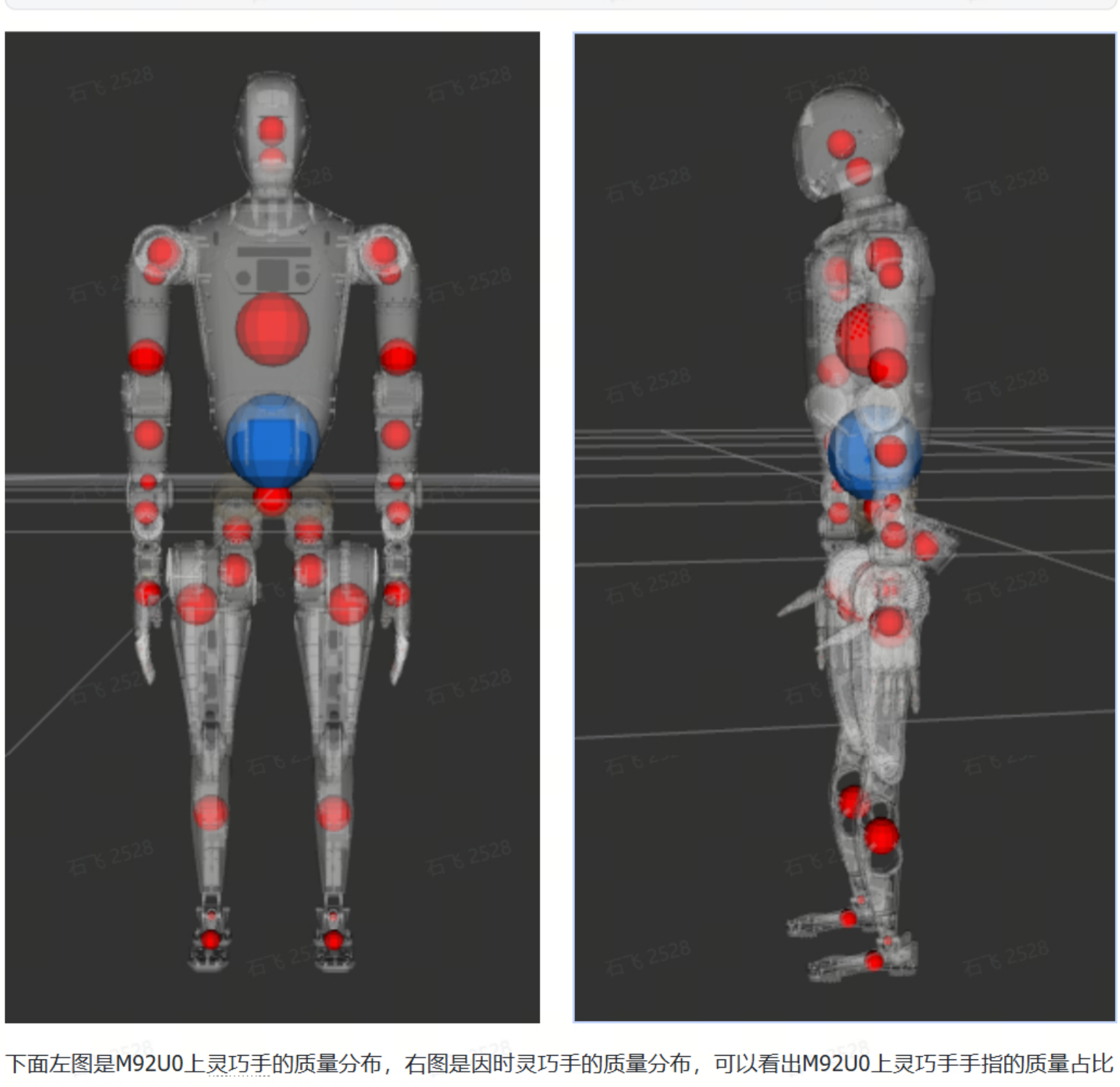
> 因为模型总质量：71.9kg 本身就是从URDF中统计的，现在还没有各关节组的实际称重数据，所以下面的质量的算法检查也就不具有参考意义。



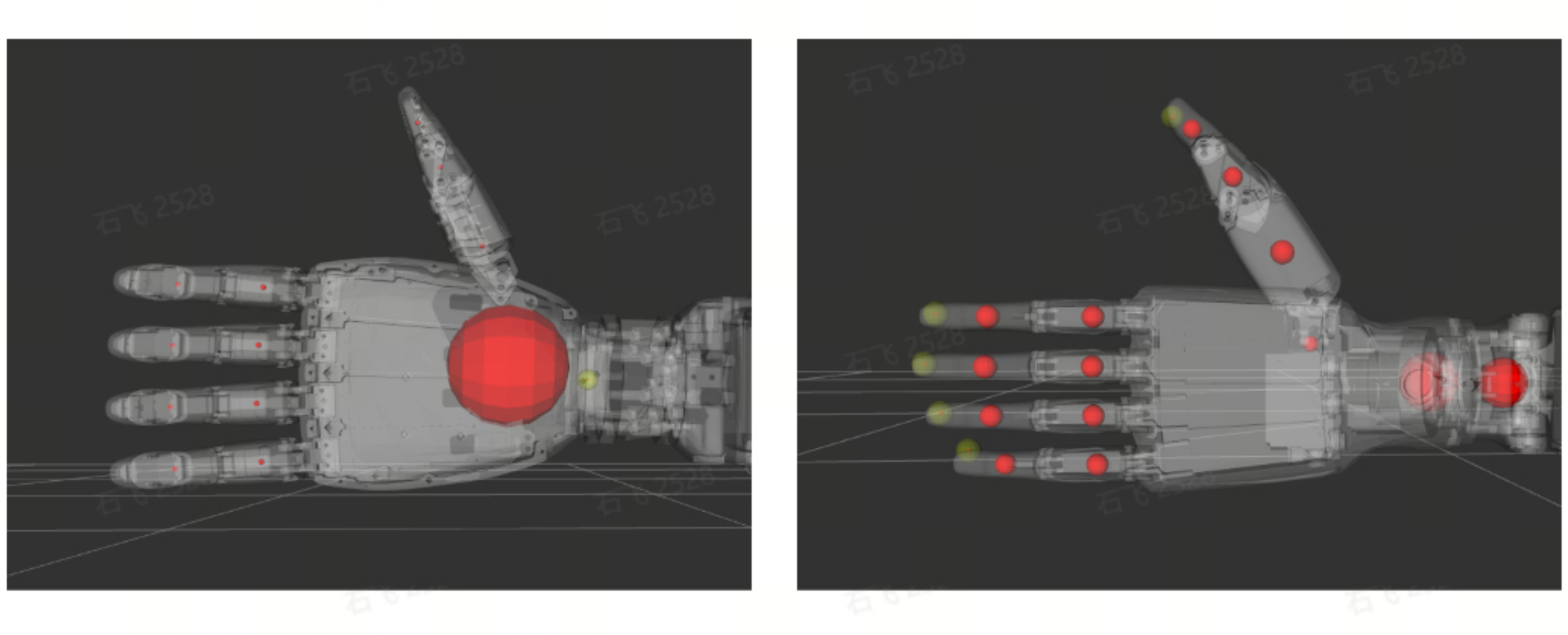
```
+-----------------------------------------------------------------------------------------+
|                              Link Mass Validation Result (M92U0)                         |
+-------+------------+-------------------+-------------------+-----------+-----------------+
| Index |   Group    | Design Value (Kg) | Actual Value (Kg) | Error (%) | Result (Pass/Fail) |
+-------+------------+-------------------+-------------------+-----------+-----------------+
|   1   |    base    |       3.3753      |       3.3753      |  0.0006   |       Pass      |
|   2   |  left_leg  |      11.1474      |      11.1474      |  0.0004   |       Pass      |
|   3   |  left_leg  |      11.1474      |      11.1474      |  0.0004   |       Pass      |
|   4   |   waist    |      24.4302      |      24.4302      |  0.0001   |       Pass      |
|   5   |    head    |       2.4767      |       2.4767      |    0.0    |       Pass      |
|   6   |  left arm  |       9.6663      |       9.6663      |  0.0002   |       Pass      |
|   7   | right arm  |       9.6663      |       9.6663      |  0.0002   |       Pass      |
|   8   | left hand  |        0.0        |       0.0012      |   100.0   |       Pass      |
|   9   | right hand |        0.0        |       0.0012      |   100.0   |       Pass      |
|  10   |   Total    |      71.9094      |       71.912      |  0.0035   |       Pass      |
+-------+------------+-------------------+-------------------+-----------+-----------------+
```

下图是M92U0的质量可视化，红色球体是各Link的质心，蓝色球体是整机重心，总体上无明显异常。

```
[center_of_mass_publisher]: Center of Mass: x=-0.001605, y=0.000258, z=0.098073
```



下面左图是M92U0上灵巧手的质量分布，右图是因时灵巧手的质量分布，可以看出M92U0上灵巧手手指的质量占比非常小，这在仿真时，容易引起碰撞效果异常。



2.2.3 灵巧手常用惯性参数估算方法

> 在修改惯性参数时，最好不要只改质量，而不更新惯性矩阵。

对于灵巧手的惯性参数，通常我们会基于 `trimesh` （或其他工具）进行估算。首先，我们会计算或实测整只手的质量，然后通过 `trimesh` 计算总的体积，并计算平均密度：

```python
import trimesh

# Load all meshes
meshes = {}
for mesh_basename in mesh_list:
    if mesh_basename in collision_mesh_basenames:
        link_name = path.splitext(mesh_basename)[0]
        mesh_path = path.join(collision_mesh_dir, mesh_basename)
        meshes[link_name] = trimesh.load(
            mesh_path, force="mesh", ignore_materials=True)
    else:
        print("The mesh file '%s' was not found in the meshes directory." % mesh_basename
        sys.exit(1)

# Compute the total volume of the robot in order to estimate the required density
total_volume = 0.0
for link_name in meshes:
    mesh = meshes[link_name]
    print("Volume estimate of %s: %f m^3" % (link_name, mesh.volume))
    total_volume += mesh.volume

# Compute average density
average_density = total_mass / total_volume
print("Average density estimate: {:.6F} kg/m^3".format(average_density))
```

最后，我们修改每一个Mesh的密度，并重新计算它的质量、质心和惯性矩阵：

```python
# Estimate inertial properties for each link
mass = {}
inertia = {}
centre_of_mass = {}
for link_name in meshes:
    mesh = meshes[link_name]
    mesh.density = average_density
    mass[link_name] = mesh.mass
    inertia[link_name] = mesh.moment_inertia
    centre_of_mass[link_name] = mesh.center_mass
```

> 这里估算的前题是手的密度均匀，实现上密度一定是不均匀的，但这点误差是在可接受范围内的，所以这也是一种常用的方法。

上面介绍的是对一组关节做估算，同样的道理，我们可以对一个 `Link` 进行计算：

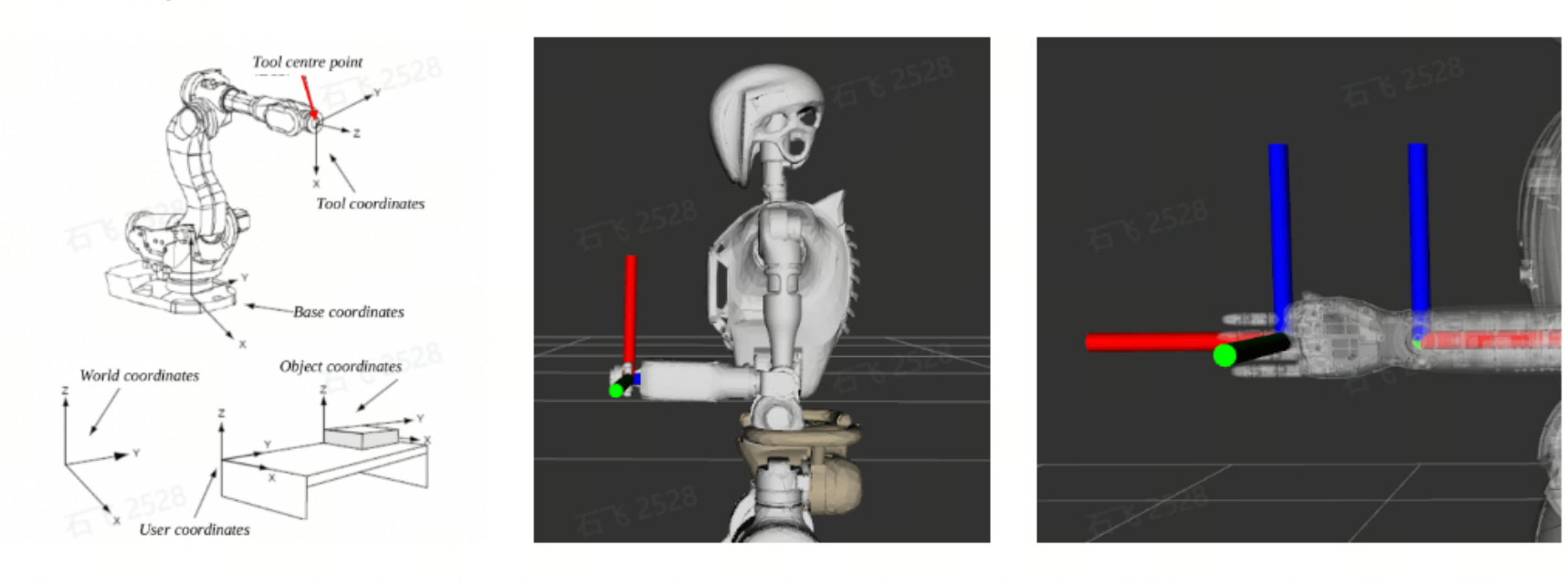
```python
# Calculate inertial properties for each link
mass = {}
inertia = {}
centre_of_mass = {}
for link_name in meshes:
    mesh = meshes[link_name]
    mesh.density = mass_data[link_name] / mesh.volume
    mass[link_name] = mesh.mass
    inertia[link_name] = mesh.moment_inertia
    centre_of_mass[link_name] = mesh.center_mass
```

3. 建议优化的内容

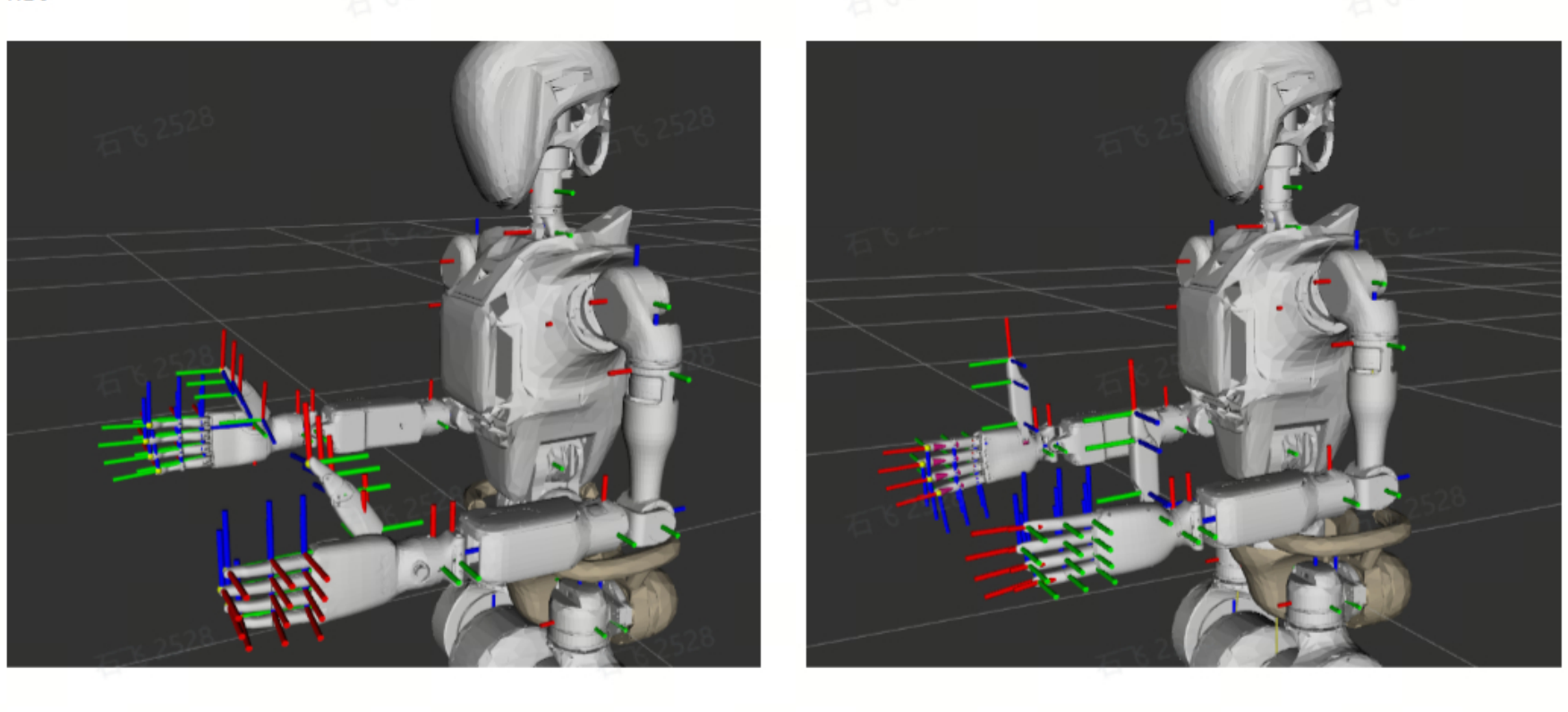
3.1 灵巧手相关坐标系

现在人形机器人可以更换多款或多型号的灵巧手已经是很普遍的现象了，所以我们通常会在手臂的末端法兰盘的位置建立坐标系，也就是我们常说的工具中心点（Tool Center Point, 简写TCP）坐标系，以方便安装、切换各种手。

为了方便抓取算法，通常我们也会在手掌的合适位置添加末端执行器坐标系（可参考末端执行器坐标系与工件坐标系的关系）。

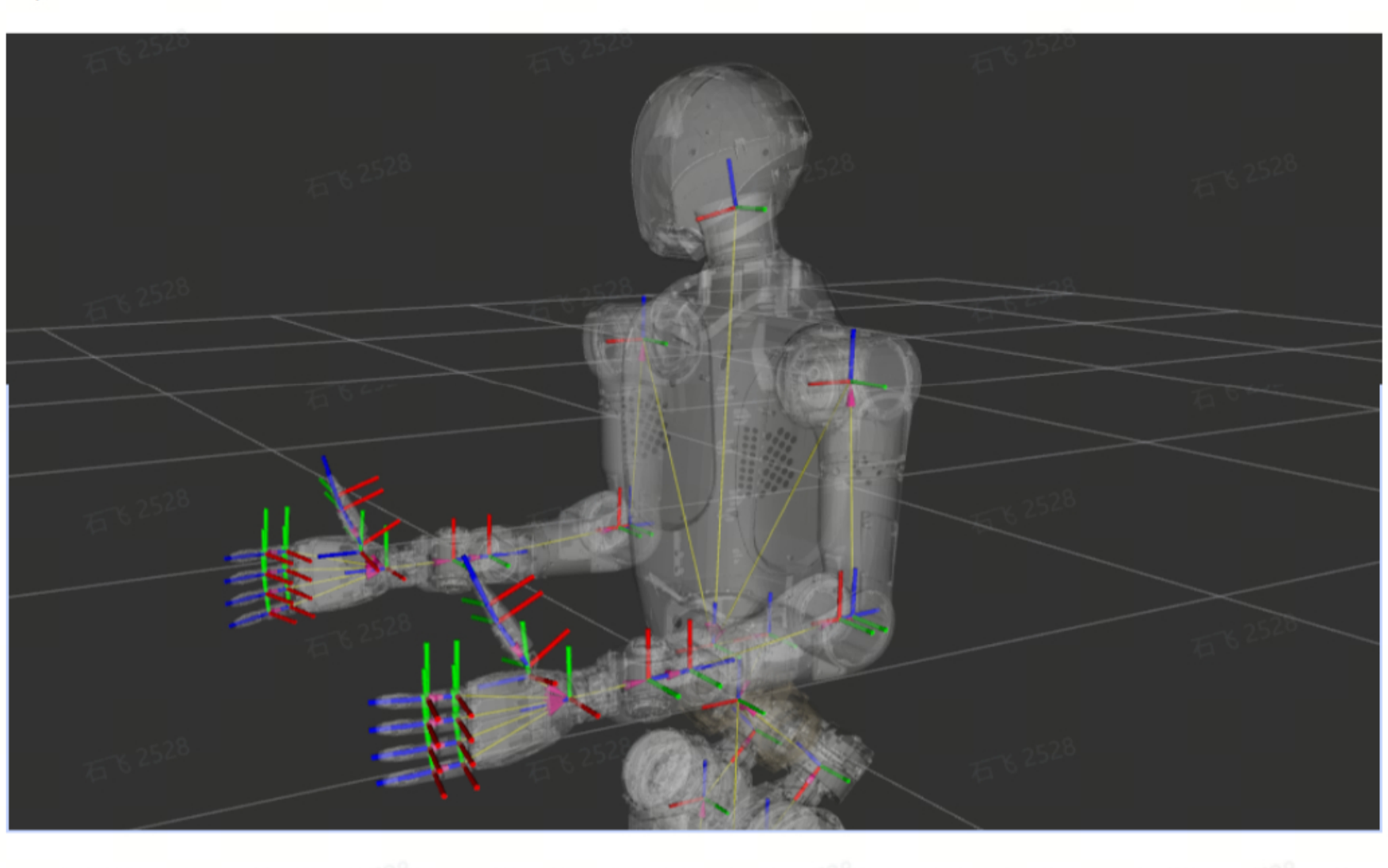


下面左图是GR1T2搭配因时的灵巧手，右图是搭配fourier的灵巧手。可以看出在TCP的基础上，切换手是很方便的。



3.2 灵巧手控制方向的问题

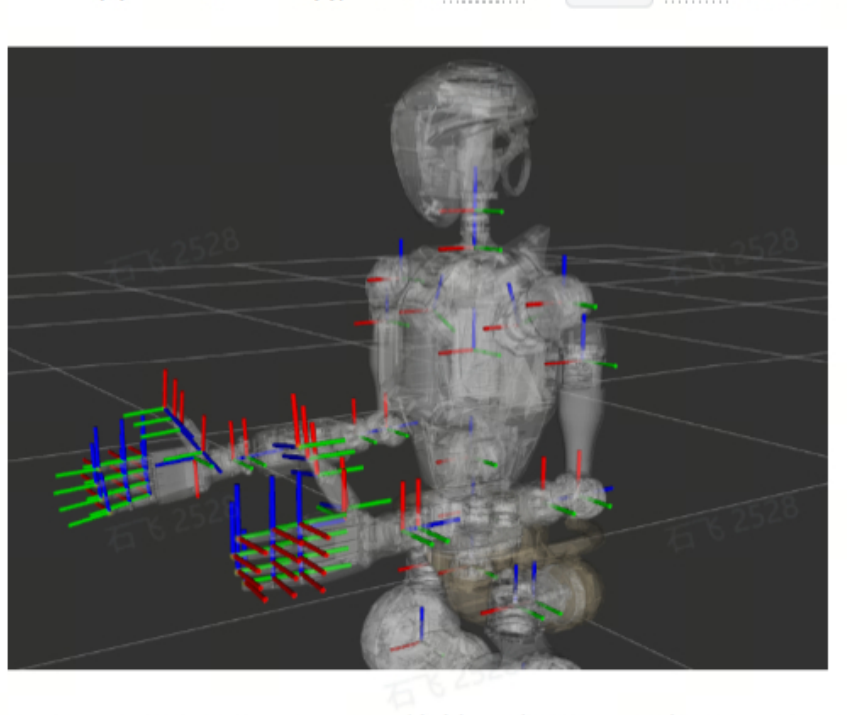
M92U0双手的坐标是一致的，这就造成左手手指反向控制（0 -> 负）时是手指内曲，右手手指正向控制（0 -> 正）时是手指内曲。

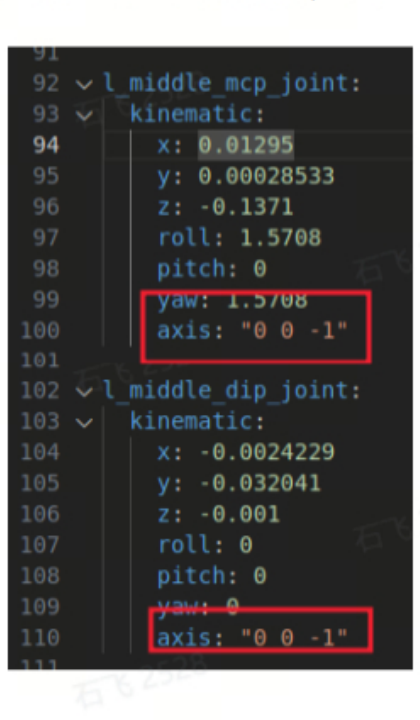


当前机器人手的控制还是延用了夹爪"Open"和"Close"的逻辑，所有手指默认姿态是舒张开的样子，此时关节位置为0，当正向控制时，手指内曲，相当于"Close"。

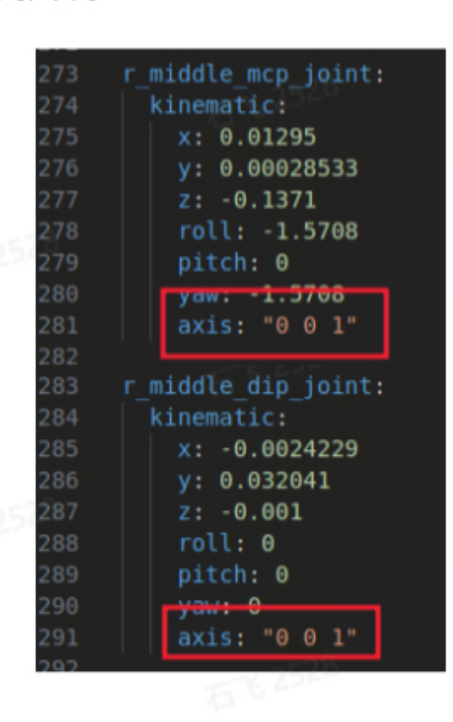
在此基础上就延伸出两种坐标系定义的方法：

- 左右手坐标系一致，但轴URDF中 `axis` 标签取反。因时的手就是这种做法，但是不推介。



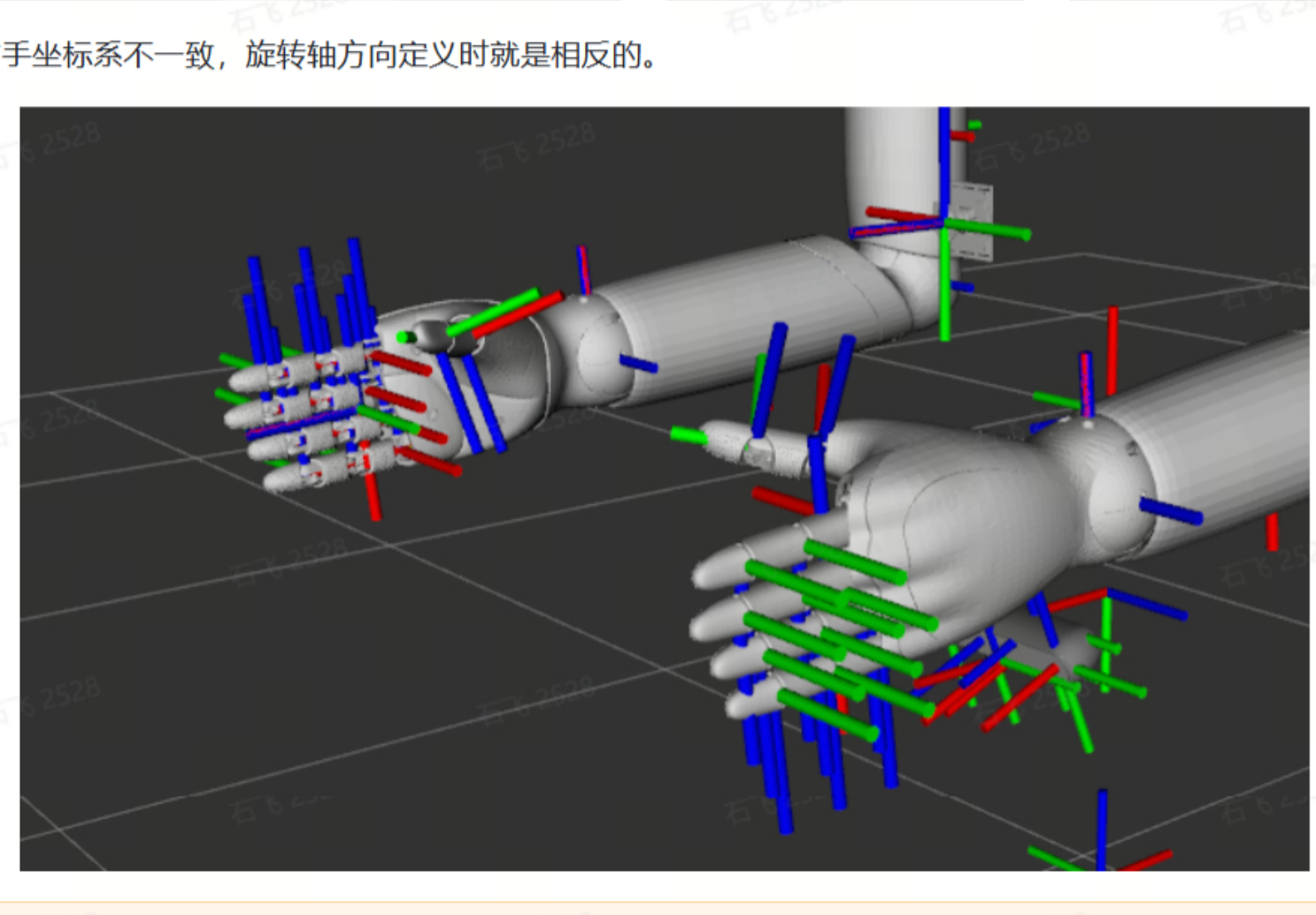


```
l_middle_mcp_joint:
  kinematic:
    x: 0.01295
    y: 0.00028533
    z: -0.1371
    roll: 1.5708
    pitch: 0
    yaw: 1.5708
    axis: "0 0 -1"

l_middle_dip_joint:
  kinematic:
    x: -0.0024229
    y: -0.032041
    z: -0.001
    roll: 0
    pitch: 0
    yaw: 0
    axis: "0 0 -1"
```



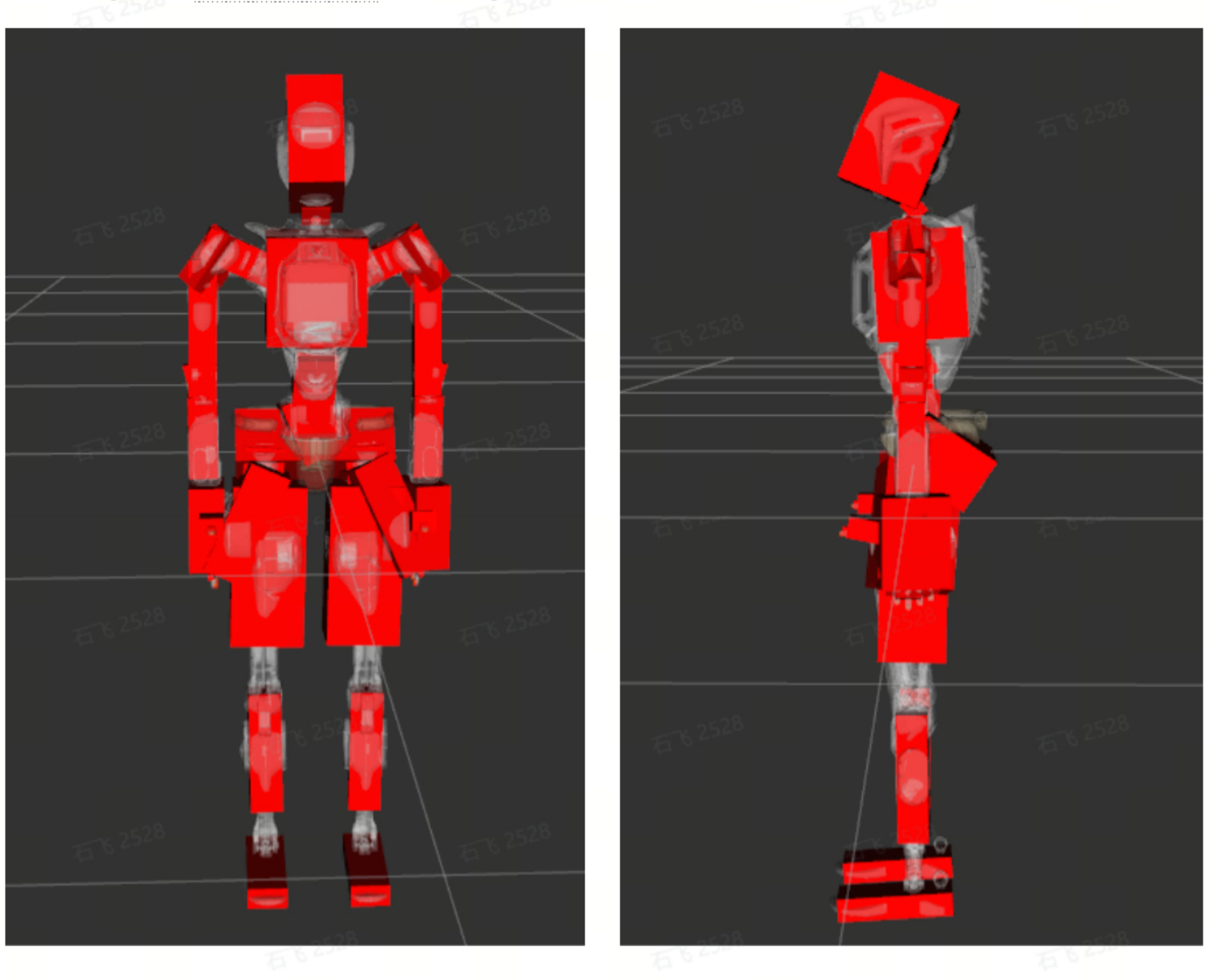
```
r_middle_mcp_joint:
  kinematic:
    x: 0.01295
    y: 0.00028533
    z: -0.1371
    roll: -1.5708
    pitch: 0
    yaw: -1.5708
    axis: "0 0 1"

r_middle_dip_joint:
  kinematic:
    x: -0.0024229
    y: 0.032041
    z: -0.001
    roll: 0
    pitch: 0
    yaw: 0
    axis: "0 0 1"
```

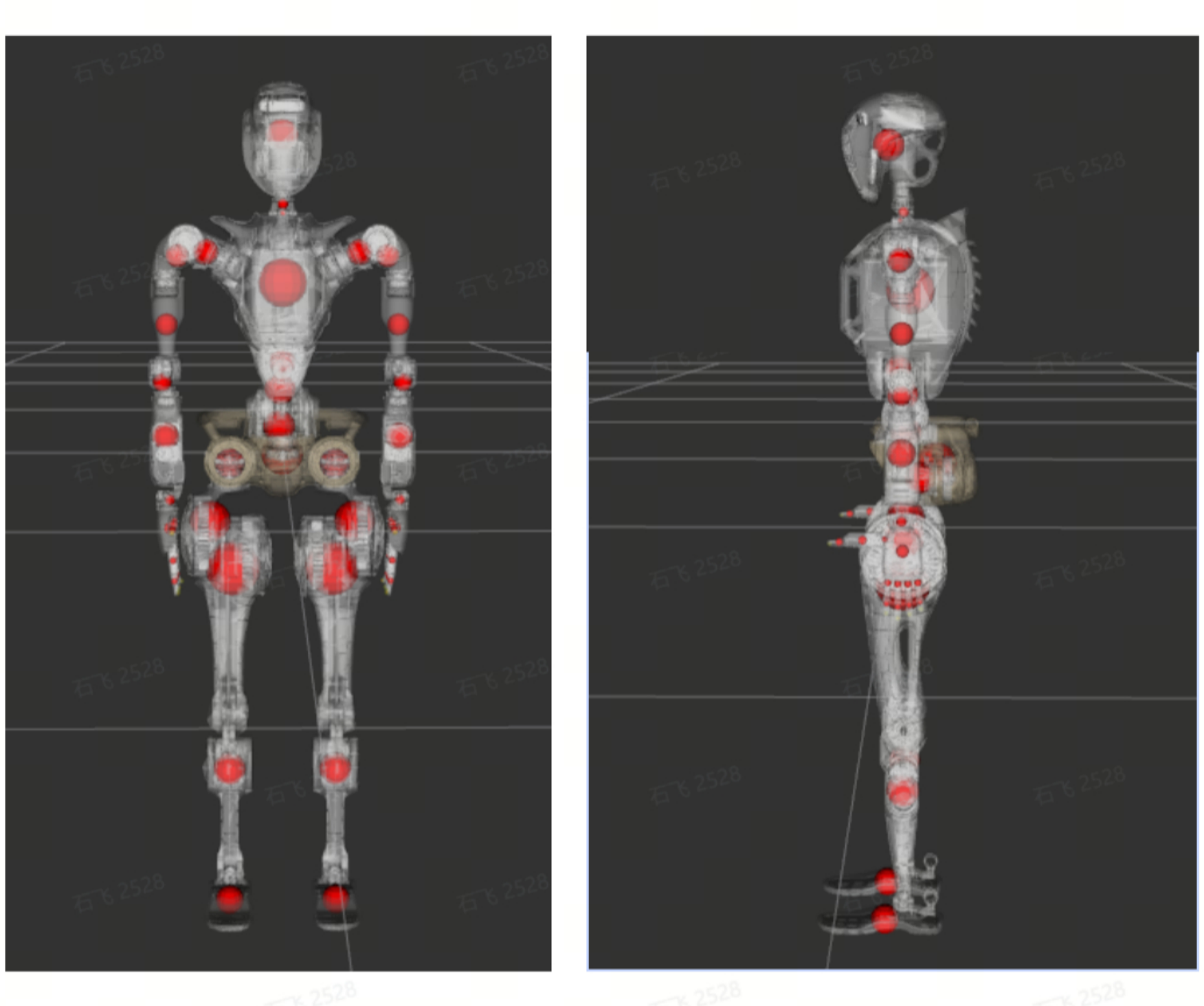
- 左右手坐标系不一致，旋转轴方向定义时就是相反的。



> 当然这也取决于灵巧手控制驱动的定义。

附录（其它人形机器人可视化）：





反向引用 (1) 本文引用 (1) 关系图
2024系统架构周报
7 人点赞
输入评论